

UNIVERSIDAD DEL VALLE DE GUATEMALA

CC3069 – Computación Paralela

Sección 10

Ing. Kimberly Barrera



Proyecto 1 - Entrega 2

Angie Nadissa Vela López, 23764

Javier Alexander Linares Chang, 231135

Roberto Emiliano Camposeco Torres, 23968

GUATEMALA, 20 de agosto de 2026

Selección de herramienta gráfica

Descripción breve de la herramienta seleccionada

Se seleccionó SDL2 (Simple DirectMedia Layer 2) como librería gráfica para el desarrollo del screensaver. SDL2 es una librería multiplataforma escrita en C que da una capa de abstracción sobre el sistema operativo para crear ventanas, manejar eventos de entrada, medir tiempo con alta resolución y renderizar en 2D con aceleración por hardware, a través de su API SDL_Renderer. A diferencia de OpenGL, SDL2 no exige armar manualmente un pipeline gráfico (shaders, buffers de vértices, matrices de proyección); ofrece primitivas de dibujo directas junto con soporte de blending (mezcla de colores y transparencias).

Justificación de su uso para el screensaver

- **Se alinea con el problema a resolver.**
 - La propuesta Neon Lines consiste en primitivas 2D simples (líneas/haces de luz) sobre un fondo oscuro; SDL2 provee estas primitivas de forma directa, sin necesidad de un pipeline 3D.
- **Se recomienda en las instrucciones del proyecto.**
 - El enunciado del proyecto sugiere SDL u OpenGL, y usa SDL2 en su propio ejemplo de referencia (screensaver de N círculos rebotando).
- **Encaja con la paralelización que planeamos.**
 - SDL no es thread-safe, lo cual encaja con el patrón de paralelización planeado: la actualización de posiciones y la detección de colisiones son paralelizables con OpenMP, mientras que el renderizado se queda secuencial en el hilo principal.
- **Su configuración es simple y práctica.**
 - OpenGL exige manejar contexto gráfico, shaders GLSL y buffers de vértices, algo que no aporta nada cuando lo que realmente se evalúa es la paralelización con OpenMP, no el renderizado.
- **Permite el efecto neón de nuestro diseño sin complicaciones.**

- El efecto de glow neón se logra dibujando cada haz con un modo de mezcla aditivo (SDL_BLENDMODE_ADD), sin necesidad de shaders personalizados.
- **Facilita medir rendimiento.**
 - SDL2 trae temporizadores de alta resolución (SDL_GetTicks, SDL_GetPerformanceCounter) que facilitan calcular FPS y tiempos de ejecución.

Requisitos básicos para su instalación y uso con C/C++

Entorno de desarrollo del equipo: Windows con MinGW-w64 (g++), sin MSYS2/pacman disponible. Pasos que seguimos:

- **Compilador.**
 - MinGW-w64 (GCC/G++) con soporte de OpenMP verificado (g++ 16.1.0, modelo de hilos posix-seh).
- **Librería SDL2.**
 - Descargamos el paquete de desarrollo precompilado para MinGW desde la página oficial de releases de SDL (SDL2-devel-<version>-mingw.tar.gz), con cabeceras (include/SDL2/*.h), librerías de enlace (lib/libSDL2.a, libSDL2main.a, libSDL2.dll.a) y la DLL en tiempo de ejecución (bin/SDL2.dll).
- **Ubicación en el proyecto.**
 - Los archivos quedaron en libs/SDL2/ dentro del repositorio, para que el build sea autocontenido y se pueda reproducir en cualquier máquina con el mismo compilador.
- **Compilación.**
 - g++ src/main.cpp -o screensaver.exe -I libs/SDL2/include -L libs/SDL2/lib -lmingw32 -lSDL2main -lSDL2 -lopenmp.
- **Ejecución.**
 - SDL2.dll debe quedar junto al ejecutable (o en el PATH) para que el programa cargue en tiempo de ejecución.

Ejemplo de referencia

Las instrucciones del proyecto incluyen la referencia de un screensaver hecho en C++ con SDL2 y OpenMP que genera N círculos que se mueven y rebotan en los bordes de la ventana, mostrando el contador de FPS en consola y en el título de la ventana. Este ejemplo es básicamente análogo a lo que proponemos con nuestro screensaver, generar N elementos independientes con movimiento, rebote y necesidad de renderizado secuencial sobre lógica que sí se puede paralelizar.

Sin embargo, cambiamos la forma geométrica de los elementos a renderizar (haces de luz en lugar de círculos) y le dimos una estética neón inspirada en la película Tron. La documentación oficial de SDL2, también incluye ejemplos equivalentes, como el patrón estándar de game loop (inicialización, poll de eventos, actualización de estado, limpieza de pantalla, renderizado, presentación de buffer) que usamos como base para la estructura del programa.

Prueba de concepto

Desarrollamos una prueba de concepto en C++ (src/main.cpp) que crea una ventana SDL2 de 800x600 píxeles y visualiza el elemento principal de la propuesta: un haz de luz neón que se mueve por la pantalla. El programa recibe N (cantidad de haces) como argumento de línea de comandos, valida ese argumento de forma defensiva y muestra el contador de FPS en el título de la ventana y en consola.

- **Creación de la ventana.**
 - Cada haz tiene un origen fijo fuera del área visible y sigue una trayectoria curva calculada de antemano; el haz "crece" revelando esa curva poco a poco en vez de trasladarse.
- **Elemento visualizado.**
 - N haces de luz con color neón (celeste y naranja, inspirados en Tron), que es el tipo de elemento definido en la propuesta original.

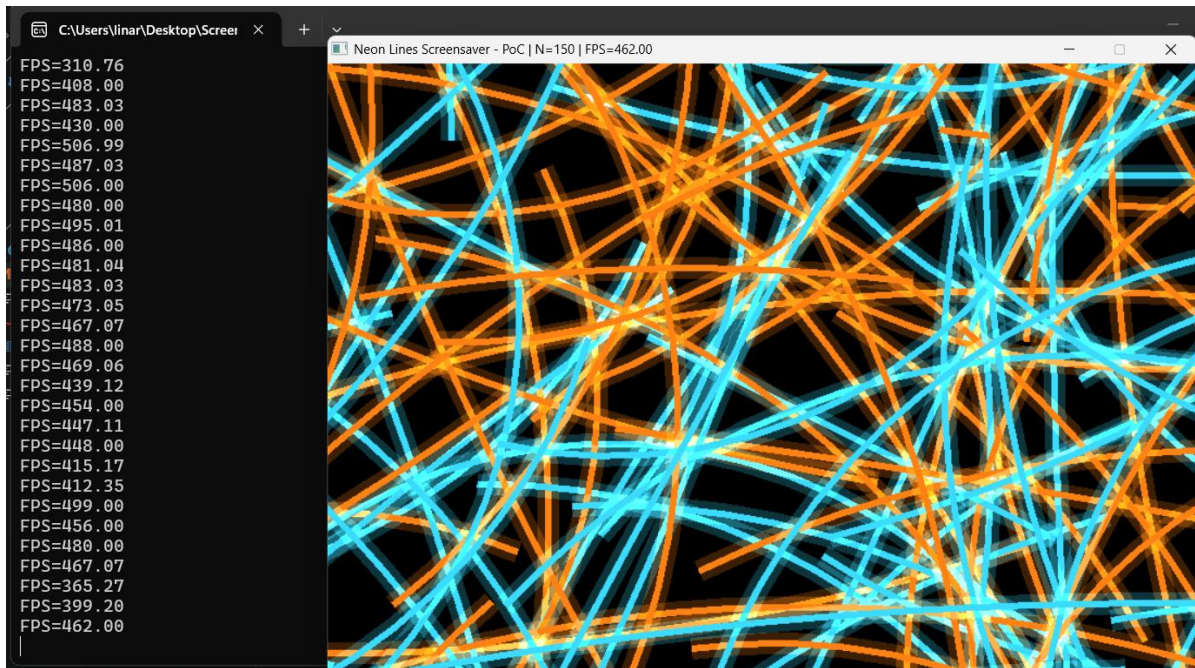


Imagen 1. Prueba de concepto en ejecución (N=150), con el registro de FPS en consola.

Repositorio

El proyecto lo hemos implementado en el siguiente [repositorio de GitHub](#).